



SDEV 2451

Full Stack Development

Frontend and Backend API Integration - 2

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

Expectations - What I expect from you

- No Late Assignments
- No Cheating
- Be a good classmate
- Don't waste your time
- Show up to class

Agenda

On the right is what we will cover today.

- Enhance the Stats Endpoint
- Add Search to the API Layer
- Update Hooks with Search and Invalidate
- Search Inputs and useEffect
- useStats Hook and StatCard Component
- Add Stat Cards to the Page
- Visualise with Recharts BarChart
- Challenge — Trips per Week and Debounce
- Key Takeaways

Enhance the Stats Endpoint

Django's ORM can compute averages and group by time periods in a single database query — no Python loops needed.

```
# blog/views.py
from datetime import timedelta
from django.db.models import Avg
from django.db.models.functions import TruncWeek
from django.utils import timezone

class BlogStatsView(APIView):
    def get(self, request):
        avg = Post.objects.aggregate(avg_words=Avg("word_count"))["avg_words"]
        six_months_ago = timezone.now() - timedelta(weeks=26)
        weekly = list(
            Post.objects
                .filter(published_at__gte=six_months_ago, word_count__isnull=False)
                .annotate(week=TruncWeek("published_at"))
                .values("week")
                .annotate(avg_words=Avg("word_count"))
                .order_by("week")
                .values_list("week", "avg_words")
        )
        return Response({
            "total_posts": Post.objects.count(),
```



Enhance the Stats Endpoint (continued)

- `Avg("word_count")` computes a database-level average — rows where `word_count` is `null` are excluded automatically
- `timezone.now() - timedelta(weeks=26)` creates a rolling 6-month window; use `timezone.now()` (not `datetime.now()`) when `USE_TZ = True`
- `TruncWeek("published_at")` truncates each timestamp to the Monday of its week — this is the `GROUP BY` column
- `.values("week").annotate(avg_words=Avg( ... ))` is Django's group-by pattern: `values()` sets the group, the second `annotate()` applies the aggregate within it

Add Search to the API Layer

Updating the API functions to accept a search parameter keeps all URL-building logic in one place — hooks and pages never construct URLs themselves.

```
// src/api/blog.js
const BASE_URL = 'http://localhost:8000/api/v1'

export async function fetchPosts(search = '') {
  const url = search
    ? `${BASE_URL}/posts/?search=${encodeURIComponent(search)}`
    : `${BASE_URL}/posts/`
  const response = await fetch(url)
  if (!response.ok) throw new Error('Failed to fetch posts')
  return response.json()
}

export async function fetchStats() {
  const response = await fetch(`${BASE_URL}/stats/`)
  if (!response.ok) throw new Error('Failed to fetch stats')
  return response.json()
}
```

Add Search to the API Layer (continued)

- `search = ''` gives the parameter a default so existing callers that omit it continue to work unchanged
- The ternary builds the URL only when `search` is truthy — avoids sending a meaningless `?search=` to the server
- `encodeURIComponent(search)` percent-encodes special characters; "React Hooks" becomes `React%20Hooks`
- `fetchStats` is a plain fetch to the stats endpoint — same pattern as the others, no parameters needed

Update Hooks with Search and Invalidate

Hooks accept a search term, include it in the `queryKey` for per-search caching, and expose an `invalidate` function the page can call explicitly.

```
// src/hooks/usePosts.js
import { useQuery, useQueryClient } from '@tanstack/react-query'
import { fetchPosts } from '../api/blog'

export function usePosts(search = '') {
  const queryClient = useQueryClient()

  const { data: posts = [], isLoading, isError, error } = useQuery({
    queryKey: ['posts', search],
    queryFn: () => fetchPosts(search),
  })

  function invalidate() {
    queryClient.invalidateQueries({ queryKey: ['posts', search] })
  }

  return { posts, isLoading, isError, error, invalidate }
}
```

Update Hooks with Search and Invalidate (continued)

- `queryKey: ['posts', search]` — React Query caches `['posts', 'react']` and `['posts', 'django']` separately; revisiting a previous term returns the cached result instantly
- `queryFn: () ⇒ fetchPosts(search)` — arrow function captures the current `search` value at call time; without it the closure would use a stale value
- `useQueryClient()` provides access to the shared cache; `invalidateQueries` marks the entry stale and triggers a background re-fetch
- The hook returns `invalidate` so the page decides *when* to invalidate — the hook itself makes no assumptions about timing

Search Inputs and useEffect

The search term state lives in the page; a `useEffect` watches the term and calls `invalidate` whenever it changes.

```
// src/pages/PostsPage.jsx
import { useState, useEffect } from 'react'
import { usePosts } from '../hooks/usePosts'

function PostsPage() {
  const [search, setSearch] = useState('')
  const { posts, isLoading, invalidate } = usePosts(search)

  useEffect(() => {
    invalidate()
  }, [search])

  return (
    <section>
      <h2 className="text-xl font-semibold mb-3">Posts</h2>
      <input
        type="text"
        placeholder="Search posts ..."
        value={search}
        onChange={(e) => setSearch(e.target.value)}
        className="input input-bordered w-full max-w-sm mb-3"
      />
    </section>
  )
}
```



Search Inputs and useEffect (continued)

- `useEffect(() => { invalidate() }, [search])` fires after every render where `search` changed — each keystroke triggers an invalidation and a fresh fetch
- The initial render also fires the effect, which is harmless — invalidating a fresh query just triggers an immediate background re-fetch
- `value={search}` and `onChange={setSearch}` make the input controlled — React is the single source of truth for its value
- The DRF `SearchFilter` backend performs a case-insensitive `icontains` — typing `"react"` matches `"React Hooks"` without needing exact case

useStats Hook and StatCard Component

A dedicated hook fetches the stats endpoint, and a reusable `StatCard` component displays a single value with a loading state.

```
// src/hooks/useStats.js
export function useStats() {
  const { data: stats, isLoading, isError, error } = useQuery({
    queryKey: ['stats'],
    queryFn: fetchStats,
  })
  return { stats, isLoading, isError, error }
}
```

```
// src/components/StatCard.jsx
function StatCard({ label, value, color }) {
  return (
    <div className={`card shadow-md ${color}`}>
      <div className="card-body">
        <p className="text-sm font-medium opacity-80">{label}</p>
        <p className="text-4xl font-bold">
          {value ?? <span className="loading loading-spinner loading-sm" />}
        </p>
      </div>
    </div>
  )
}
```



useStats Hook and StatCard Component (continued)

- `queryKey: [ 'stats' ]` is stable — there is no search parameter — so stats are fetched once and cached until invalidated
- `StatCard` receives `color` as a prop (e.g. `"bg-primary text-primary-content"`) so the same component renders in different DaisyUI theme colours
- `value ?? <spinner />` uses nullish coalescing — the spinner only shows for `null` or `undefined`; a value of `0` renders correctly as the number `0`
- Separating the hook from the component means the stat card can be reused anywhere without knowledge of the API

Add Stat Cards to the Page

A configuration array and a responsive grid turn adding or reordering cards into a one-line change.

```
// src/pages/PostsStatsPage.jsx
import StatCard from '../components/StatCard'
import { useStats } from '../hooks/useStats'

const STAT_CARDS = [
  { key: 'total_posts',    label: 'Total Posts',    color: 'bg-primary text-primary-content' },
  { key: 'total_authors', label: 'Total Authors',  color: 'bg-secondary text-secondary-content' },
  { key: 'avg_word_count', label: 'Avg Word Count', color: 'bg-accent text-accent-content' },
]

function PostsStatsPage() {
  const { stats } = useStats()
  return (
    <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-4">
      {STAT_CARDS.map(({ key, label, color }) => (
        <StatCard key={key} label={label} value={stats?.[key]} color={color} />
      ))}
    </div>
  )
}
```

Add Stat Cards to the Page (continued)

- `STAT_CARDS` is defined outside the component so it is not recreated on every render
- `stats?.[key]` uses optional chaining — safely returns `undefined` while `stats` is still loading, which causes `StatCard` to render the spinner via `??`
- `useStats` is called once; both the cards and any chart on the same page share the single cached response — only one network request to `/api/v1/stats/`
- `lg:grid-cols-3` and `sm:grid-cols-2` make the layout responsive: 3 columns on large screens, 2 on medium, 1 on mobile

Visualise with Recharts BarChart

Recharts components are composable — wrap any chart in `ResponsiveContainer` and it fills its parent at any screen width.

```
// src/components/WeeklyStatsChart.jsx
import { BarChart, Bar, XAxis, YAxis, CartesianGrid, Tooltip, ResponsiveContainer } from 'recharts'

function WeeklyStatsChart({ data }) {
  const chartData = data.map((entry) => ({
    week: new Date(entry.week).toLocaleDateString('en-CA', { month: 'short', day: '2-digit' }),
    avg_words: entry.avg_words,
  }))

  return (
    <div className="card bg-base-100 shadow-md">
      <div className="card-body">
        <h3 className="card-title text-base">Avg Word Count per Week</h3>
        <ResponsiveContainer width="100%" height={300}>
          <BarChart data={chartData}>
            <CartesianGrid strokeDasharray="3 3" />
            <XAxis dataKey="week" tick={{ fontSize: 12 }} />
            <YAxis unit="w" width={60} tick={{ fontSize: 12 }} />
            <Tooltip formatter={(v) => [`${v} words`, 'Avg']} />
            <Bar dataKey="avg_words" fill="#570df8" radius={[4, 4, 0, 0]} />
          </BarChart>
        </ResponsiveContainer>
      </div>
    </div>
  )
}
```



Visualise with Recharts BarChart (continued)

- `ResponsiveContainer width="100%"` makes the chart fill its container; `height={300}` sets a fixed pixel height
- `XAxis dataKey="week"` maps the formatted date string to each bar's label on the horizontal axis
- `YAxis unit=" w"` appends a unit to every tick; `width={60}` prevents the label from being clipped at the left edge
- `Tooltip formatter` customises the popup label so it reads `"1450 words – Avg"` rather than the raw key name
- `radius={[4, 4, 0, 0]}` rounds only the top corners of each bar — standard visual convention
- Guard the component with `data?.length > 0` so it only renders once the API has returned data

Challenge — Trips per Week and Debounce

1. Add a total posts per week chart

- Add a `posts_per_week` key to `BlogStatsView` using `TruncWeek` and `Count` over the same 6-month window
- Create a `PostsPerWeekChart` component using `Recharts LineChart` instead of `BarChart`
- Render it below the bar chart on the stats page

2. Debounce the search input

- Add `useState` for `debouncedSearch` that updates 300ms after `search` changes using `setTimeout` / `clearTimeout` inside a `useEffect`
- Pass `debouncedSearch` to `usePosts` instead of the raw `search` value — the hook only fetches when the debounced value changes
- Verify in the browser network tab that requests fire only after you stop typing, not on every keystroke

Key Takeaways

- **Avg and TruncWeek** — Django ORM aggregations compute in the database; group-by uses `.values( ... ).annotate( ... )` pattern
- **Rolling time window** — `timezone.now() - timedelta(weeks=26)`; always `timezone.now()` with `USE_TZ = True`
- **Dynamic queryKey** — include the search term so each unique search gets its own cache entry; re-visiting a term is instant
- **invalidate + useEffect** — expose `invalidate` from the hook, call it in a `useEffect` in the page; makes cache invalidation explicit and teachable
- **encodeURIComponent** — always encode user input in URLs to handle spaces and special characters safely
- **Config arrays for repeated components** — `STAT_CARDS = [{ key, label, color }]` outside the component makes adding/reordering a one-line change



ESSENTIAL
TO ALBERTA





Example

Let's go do an example together

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS